

RSA Factorization Attacks: Experimental Evaluation of Trial Division, Fermat, and Pollard's Rho

Christopher Alejandro Ortiz Bustillos
School of Math. and Computational Sciences
Universidad Yachay Tech
Urcuqui, Ecuador

Abstract—The security of the RSA cryptosystem relies heavily on the computational intractability of factoring large composite moduli. This paper presents an experimental evaluation of three classical factorization algorithms—Trial Division, Fermat Factorization, and Pollard's Rho—against small RSA moduli. The implementation successfully recovers the prime factors p and q , allowing for the reconstruction of the private key d and the decryption of intercepted ciphertexts. Our experimental benchmark analyzes execution times across various bit sizes and explores the specific mathematical vulnerability of Fermat's algorithm when primes are closely distributed.

Index Terms—RSA, Cryptography, Factorization, Pollard's Rho, Fermat Factorization, Trial Division

I. INTRODUCTION

The RSA public-key cryptosystem is foundational to modern digital security. Its public key consists of a modulus n and an exponent e , where $n = p \times q$ is the product of two large, secret primes. The security of RSA collapses if an attacker can efficiently factor n to recover p and q , which subsequently allows the computation of Euler's totient function $\phi(n)$ and the extraction of the private key d .

This technical report details the implementation and benchmarking of three distinct factorization attacks. The objective is to empirically assess their asymptotic behavior and practical viability against small to medium-sized moduli, highlighting the mathematical conditions under which specific algorithms excel or fail.

II. METHODOLOGY AND FACTORIZATION ALGORITHMS

A. Trial Division

Trial Division is a deterministic brute-force approach that searches for an exact divisor of n . To optimize the search space, the algorithm only tests odd integers up to $\sqrt{n}$. While statistically inefficient for properly generated RSA moduli, its linear progression makes it highly effective if one of the factors is unusually small.

B. Fermat Factorization

Fermat's algorithm exploits the mathematical identity $n = a^2 - b^2 = (a - b)(a + b)$. It initializes the search at $a = \lceil \sqrt{n} \rceil$ and iteratively increments a until $b^2 = a^2 - n$ forms a perfect square. The prime factors are then extracted as $p = a - b$

and $q = a + b$. This algorithm is exceptionally fast when the distance $|p - q|$ is minimal.

C. Pollard's Rho

Pollard's Rho is a probabilistic integer factorization algorithm that utilizes a pseudo-random sequence, typically $f(x) = (x^2 + 1) \pmod{n}$, combined with Floyd's cycle-finding algorithm. It continuously evaluates the greatest common divisor $d = \gcd(|x - y|, n)$. If $1 < d < n$, a non-trivial factor is discovered. This approach achieves a time complexity of $O(n^{1/4})$, providing a significant advantage for factoring numbers with widely spaced prime factors.

D. Automated Validation

Before experimental benchmarking, the cryptographic pipeline was isolated and subjected to strict automated assertions. The test suite validated the Extended Euclidean Algorithm ($\gcd(48, 18) = 6$), the modular inverse function ($17^{-1} \pmod{3120} = 2753$), and the accurate recovery of the private key for a test modulus $n = 3233$, confirming consistency in a full encryption and decryption cycle.

III. EXPERIMENTAL RESULTS

The experimental benchmark was executed on a Cygwin POSIX environment over Windows 11. The algorithms were compiled using g++ with the `-std=c++17` standard and `-O3` optimization. Execution times were recorded using the `<chrono>` library, and each reported value represents the average of three independent runs.

A. Benchmark Across Bit Sizes

Table I presents the execution time (in milliseconds) of the three algorithms across four different moduli configurations ranging from 33 to 45 bits.

TABLE I
AVERAGE EXECUTION TIME (MILLISECONDS) ACROSS ALGORITHMS

Bits	Modulus (n)	Trial Div.	Fermat	Pollard's Rho
33	5234636191	0.058	11,344.41	8.39
40	1099511627791	1.503	48,090.41	166.19
45	3518437208891	8.020	42,395.34	1,807.34
45	35184371900411	8.020	55,130.41	796.90

B. Fermat Iteration Analysis

To evaluate the mathematical vulnerabilities of Fermat's Factorization, we analyzed two 45-bit moduli. Table II illustrates the correlation between the absolute distance $|p - q|$ and the number of iterations required to find the perfect square.

TABLE II
FERMAT ANALYSIS: DISTANCE VS. ITERATIONS (45-BIT MODULI)

Modulus (n)	Distance $ p - q $	Total Iterations
35184372088891	4,012,248,198	6,390,790,740
35184371900411	222,037,690	8,584,722,388

IV. DISCUSSION AND ANALYSIS

1) *Impact of Modulus Size on Trial Division:* The experimental data clearly demonstrates that Trial Division scales linearly up to the $\sqrt{n}$ boundary. While the time increased from 0.058 milliseconds (33 bits) to 8.020 milliseconds (45 bits), Trial Division unexpectedly outperformed the other algorithms for the generated 45-bit moduli. This statistical anomaly highlights the algorithm's extreme efficiency when a composite number happens to have one exceptionally small prime factor.

2) *Efficiency of Fermat with Close Primes:* Fermat's Factorization relies entirely on the premise that p and q are positioned near $\sqrt{n}$. When the distance $|p - q|$ is minimal, the algorithm requires negligible iterations. Conversely, our experimental data proved its weakness: with prime distances in the hundreds of millions, Fermat collapsed, requiring between 6 and 8.5 billion iterations and stalling execution for upwards of 55 seconds (approx. 55,130 ms).

3) *The Role of the GCD in Pollard's Rho:* The Greatest Common Divisor is the core collision detector of Pollard's Rho. Instead of verifying direct divisibility, the algorithm processes a pseudo-random trajectory. The GCD successfully extracts a non-trivial factor ($1 < d < n$) by identifying algebraic commonalities between the generated differences $|x - y|$ and the modulus n , bypassing the need to guess the exact prime factor.

4) *Performance on the Largest Modulus:* Against expectations for cryptographic distributions, Trial Division registered the absolute fastest time (8.020 milliseconds) for the 45-bit moduli, decisively beating Pollard's Rho (which averaged between 796 and 1,807 milliseconds) and thoroughly dominating Fermat Factorization.

5) *Breaking the RSA Private Key:* Recovering p and q fundamentally breaks RSA because it reveals the hidden mathematical structure of the modulus. With the prime factors known, an attacker calculates Euler's totient $\phi(n) = (p - 1)(q - 1)$. Applying the Extended Euclidean Algorithm to the public exponent e against $\phi(n)$ directly yields the private key d , stripping the cryptosystem of its security.

V. CONCLUSION

This experimental laboratory validated the theoretical vulnerabilities of small-scale RSA keys. The automated tests confirmed the strict mathematical reliability of the cryptographic pipeline. The experimental benchmarks demonstrated that while Pollard's Rho offers probabilistic consistency, algorithms like Fermat and Trial Division are highly sensitive to the specific arithmetic distribution of the underlying prime factors.